

Constraint Satisfaction Problems

Jacopo Mauro

Slide Based on Slides of the Artificial Intelligence: A Modern Approach book

Imperative vs. Declarative Approaches

Goal: find a goal or maximize/minimize something

Imperative:

- Explicitly describes how to compute a solution
- Requires designing and implementing a search strategy
- Example Sudoku: write a search algorithm to check every possibility

Declarative:

- Specifies what constraints a solution must satisfy
- Solver automatically finds feasible solutions
- Example Sudoku: just state the constraints (e.g., "Each row, column, and block contains 1–9 exactly once")

Defining Constraint Satisfaction Problems

A constraint satisfaction problem (CSP) consists of three components, X , D , and C :

- X is a set of variables, $\{X_1, \dots, X_n\}$.
- D is a set of domains, $\{D_1, \dots, D_n\}$, one for each variable
- C is a set of constraints that specify allowable combination of values

CSPs deal with assignments of values to variables.

A complete assignment is one in which every variable is assigned a value, and a solution to a CSP is a consistent, complete assignment.

A partial assignment is one that leaves some variables unassigned.

Partial solution is a partial assignment that is consistent

Constraint satisfaction problems (CSPs)

Search problem:

- state is defined by variables X_i with values from domain D_i
- goal test is a set of constraints specifying

Allows useful general-purpose algorithms with more power than standard search algorithms

Example: Map-Coloring

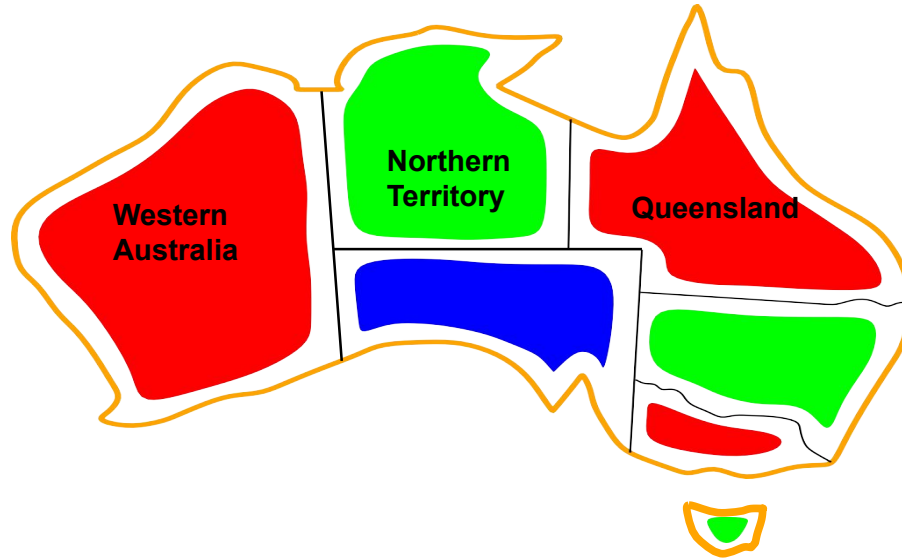


Variables WA,NT,Q,NSW,V,SA,T Domains $D_i = \{\text{red, green, blue}\}$

Constraints: adjacent regions must have different colors

e.g., $WA \neq NT$ or $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \dots\}$

Example: Map-Coloring contd.



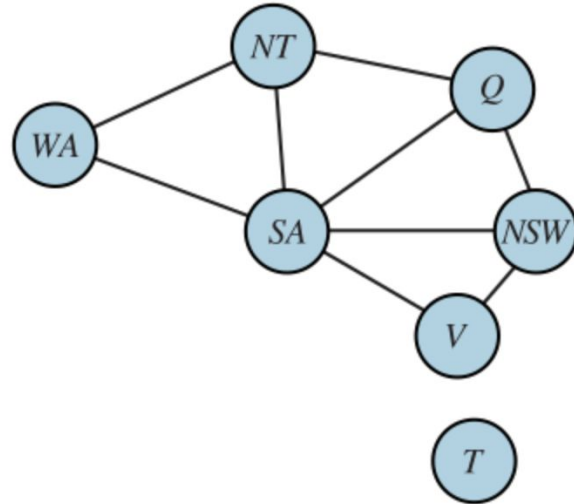
Solutions are assignments satisfying all constraints

E.g., {WA = red, NT = green, ...}

Constraint graph

Binary CSP: each constraint relates at most two variables

Constraint graph: nodes are variables, arcs show constraints



Real-world CSPs

Puzzles (Sudoku)

Assignment problems (e.g., who teaches what class)

Timetabling problems (e.g., which class is offered when and where?)

Hardware configuration

Transportation/Factory/... scheduling

Standard search formulation

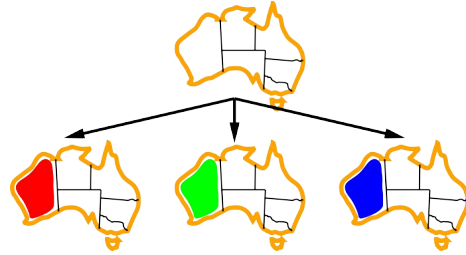
Idea

- Initial state: the empty assignment
- Successor function: assign a value to an unassigned variable that does not conflict with current assignment
- If we assigned all variables $\rightarrow$ problem solved
- If there is a variable we can not assigned $\rightarrow$ backtrack and change value
- If we backtrack to the first variable $\rightarrow$ problem is not satisfiable

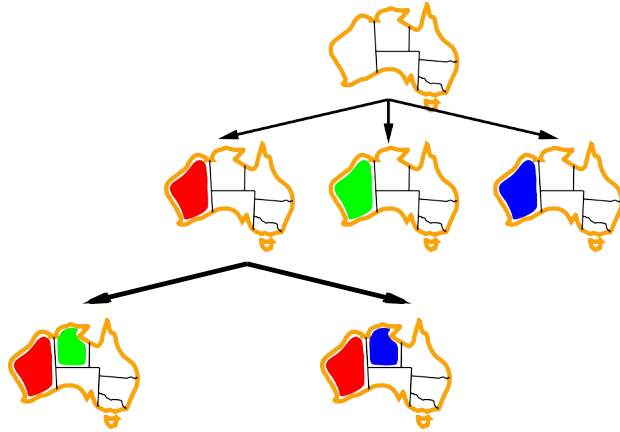
Backtracking example



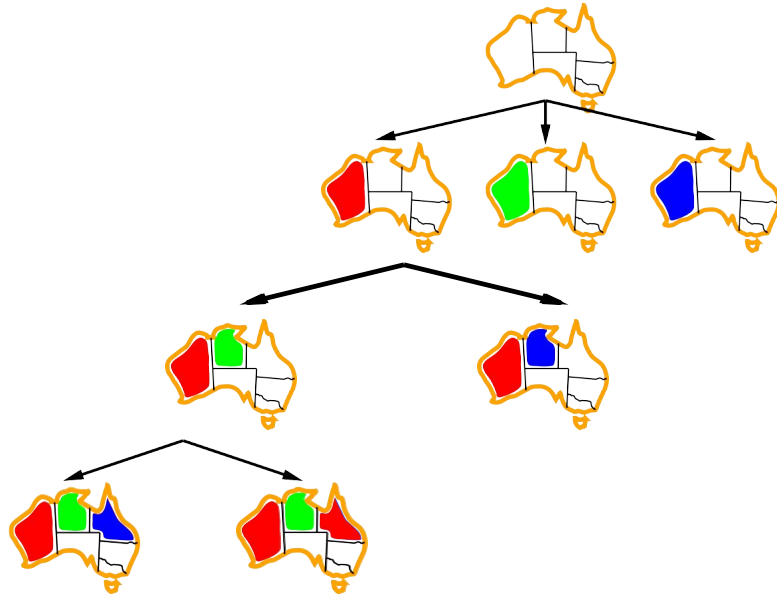
Backtracking example



Backtracking example



Backtracking example



Backtracking

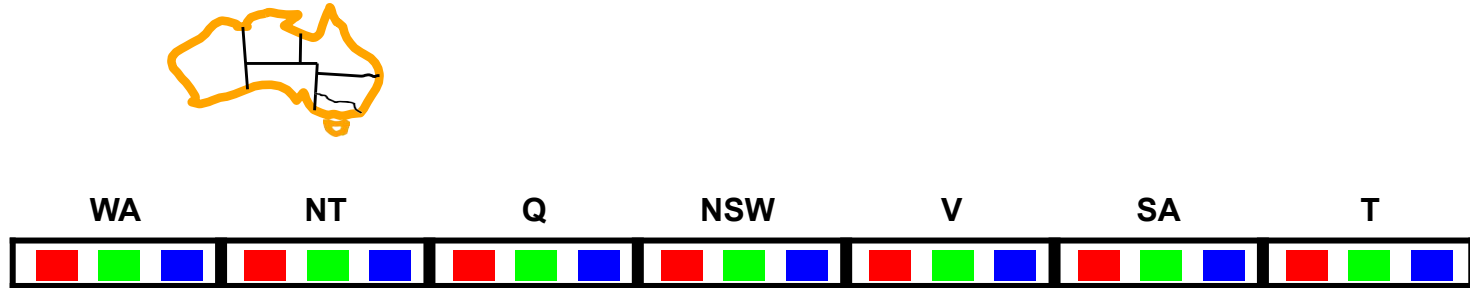
```
function BACKTRACKING-SEARCH(csp) returns a solution or failure  
  return BACKTRACK(csp, { })  
  
function BACKTRACK(csp, assignment) returns a solution or failure  
  if assignment is complete then return assignment  
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)  
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do  
    if value is consistent with assignment then  
      add {var = value} to assignment  
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)  
      if inferences  $\neq$  failure then  
        add inferences to csp  
        result  $\leftarrow$  BACKTRACK(csp, assignment)  
        if result  $\neq$  failure then return result  
        remove inferences from csp  
        remove {var = value} from assignment  
  return failure
```

Improving backtracking efficiency

General-purpose methods can give huge gains in speed:

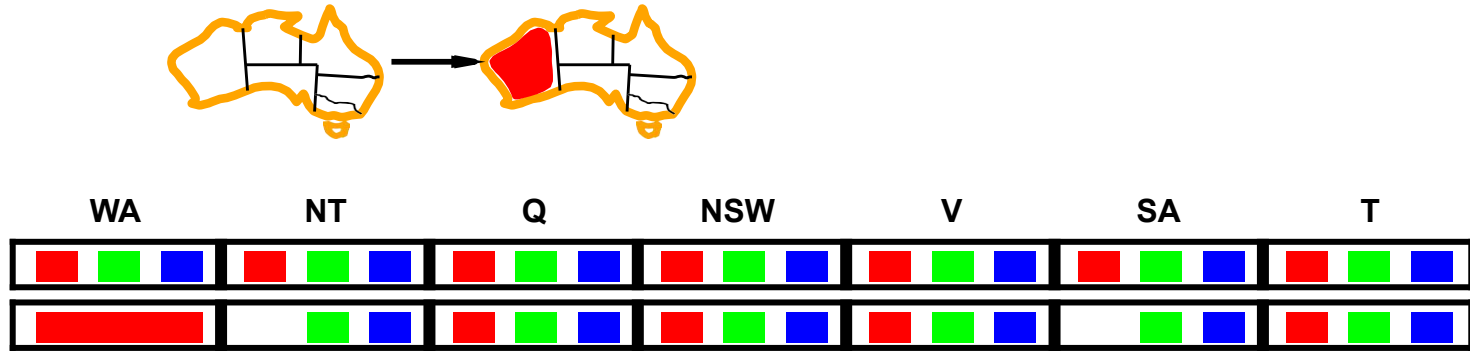
- Can we detect inevitable failure early?
- Which variable should be assigned next?
- In what order should its values be tried?

Forward checking

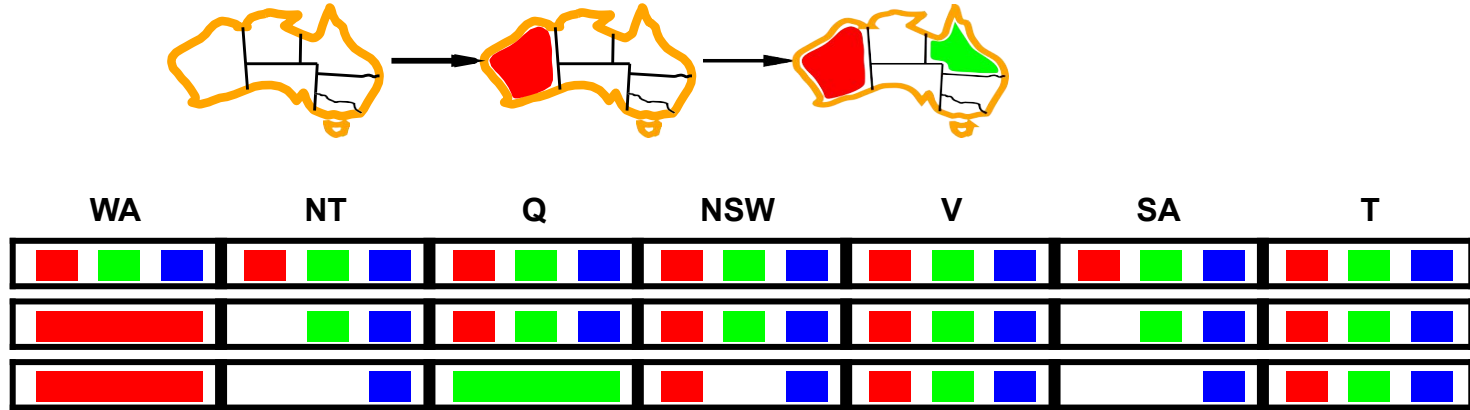


Idea: Keep track of remaining legal values for unassigned variables + terminate search when any variable has no legal values

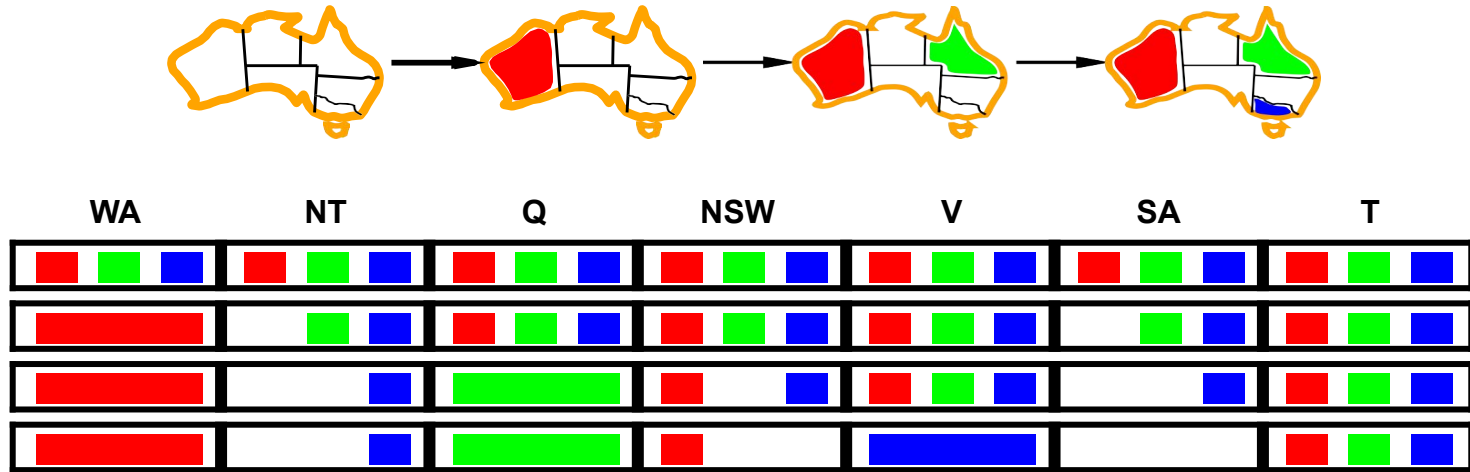
Forward checking



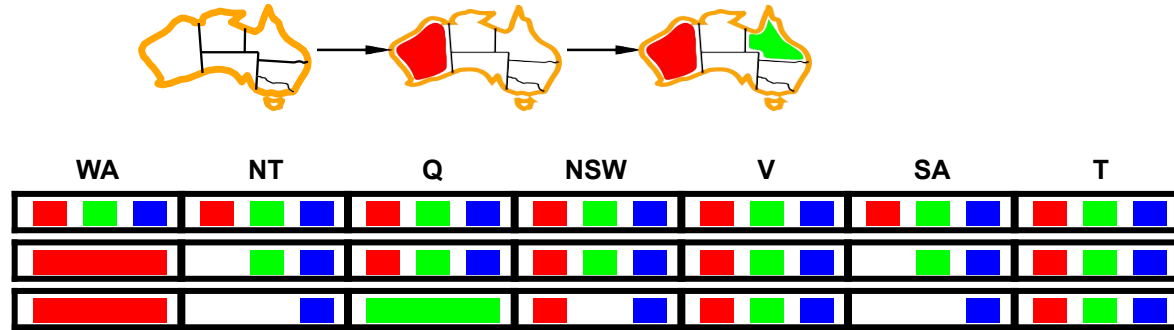
Forward checking



Forward checking



Constraint propagation



Forward checking doesn't provide early detection for all failures.

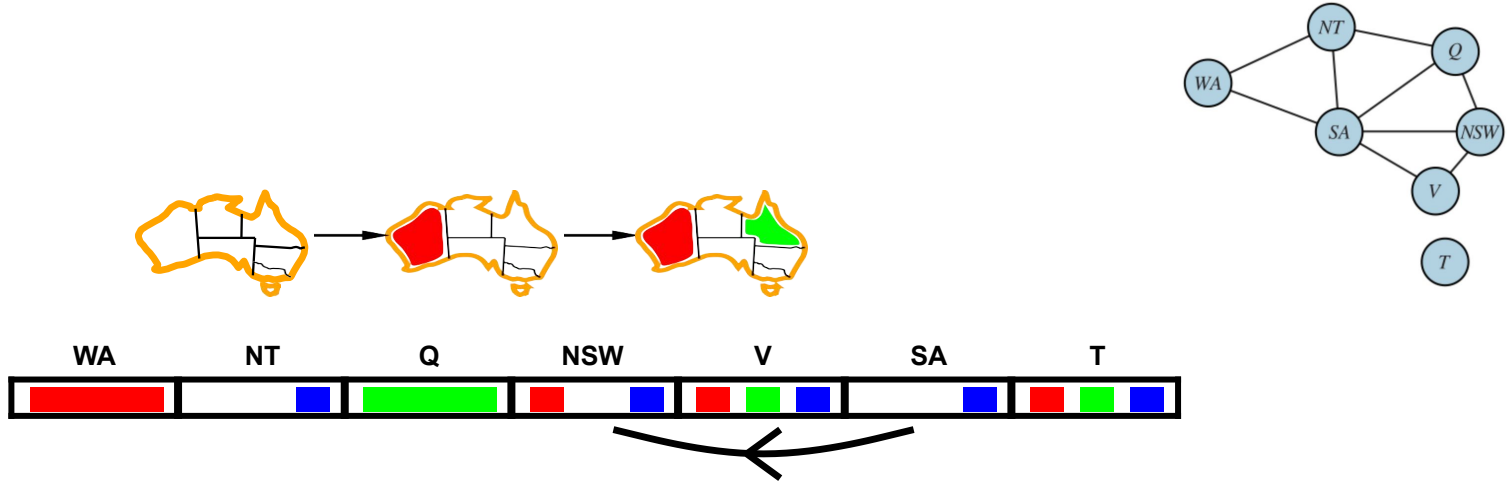
Constraint propagation repeatedly enforces constraints locally

- NT and SA cannot both be blue!

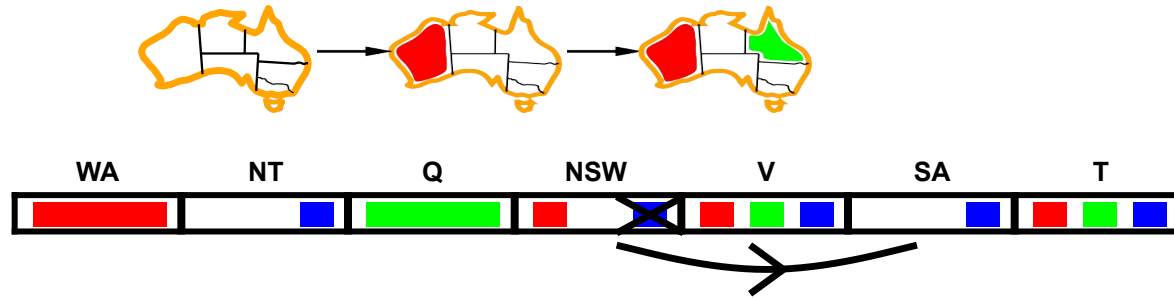
Arc consistency

Simplest form of propagation makes each arc consistent

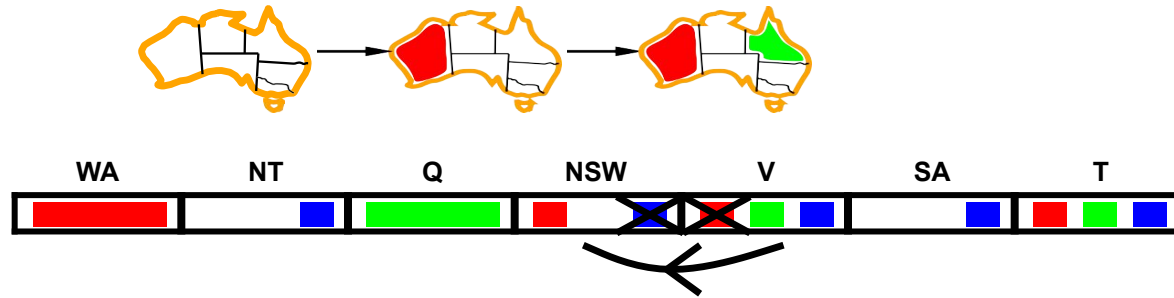
- $X \rightarrow Y$ is consistent iff for every value x of X there is some allowed y for Y



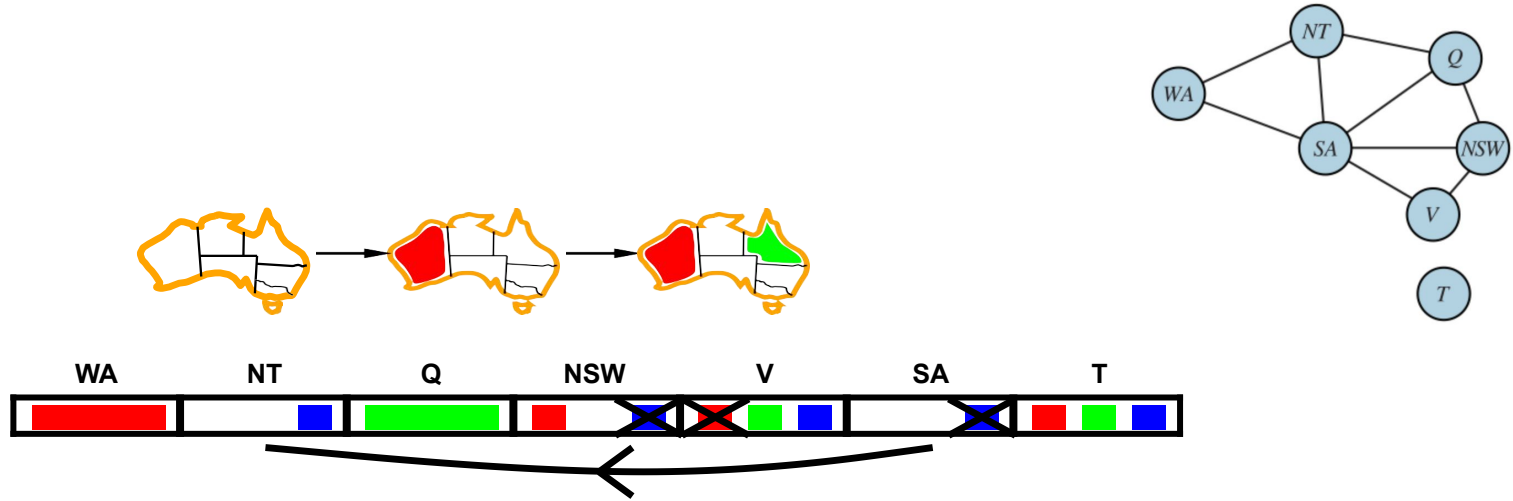
Arc consistency



Arc consistency



Arc consistency



Arc consistency detects failure earlier than forward checking

Can be run as a preprocessor or after each assignment

Local consistencies

Node Consistency (NC)

- Ensures each variable satisfies its unary constraints (i.e., constraints involving only itself)
- Example: “must start after 10 AM” removes invalid times from its domain

Arc Consistency (AC)

- Ensures that for every variable-value pair, there exists at least one valid value in every connected variable.

Path Consistency (PC)

- Ensures that if two variables satisfy a constraint, a third variable can still take a consistent value.
- Example: In a meeting room allocation, if Alice can meet in Room A or B, and Bob can only meet in Room B, then Carol must be able to meet in Room B to ensure feasibility

k-Consistency (Generalized Consistency)

- Extends path consistency to k variables.
- Example: In graph coloring, 3-consistency ensures that given two assigned colors, the third connected node can still be colored properly.

Variable heuristic

Minimum remaining values:

- Pick variable with the smallest domain
- Quickly reduces search space

Degree Heuristic

- Pick variable affecting the most constraints
- Maximizes propagation

Adaptive strategy: Dom/Wdeg (Domain over Weighted Degree)

- Weighted Degree = Sum of constraint weights, based on past failures
- Choose the variable that minimizes Domain Size / Weighted Degree

Value heuristic

Least Constraining Value (LCV)

- Choose the value that eliminates the fewest options for other variables
- Keeps more flexibility in the search space

Most Constraining Value

- Pick the value that will rule out more other values using the inference
- Helps detect failure earlier

Min-Value / Max-Value

- Strategy: Pick the smallest (or largest) value first
- Useful when values have a natural order (e.g., cost minimization)

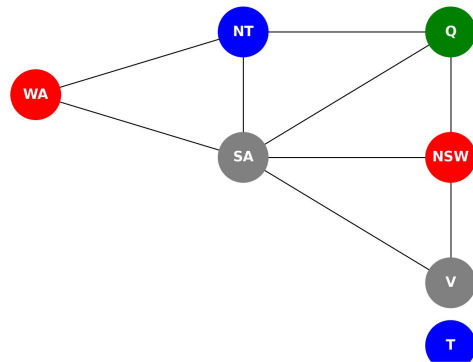
Backjumping

Improved form of backtracking that jumps back multiple levels in tree

- analyzes why the failure occurred and jumps directly to the most relevant earlier decision

Example:

- $WA = R, NSW = R, NT = B, T = B, Q = G, SA = ?$
- No possible solution with $WA = NSW$
- Backtracking will try to change T
- Backjumping avoids to explore useless assignments



Local Search of CSP

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or *failure*

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*

for $i = 1$ to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES

value $\leftarrow$ the value v for *var* that minimizes CONFLICTS(*csp*, *var*, v , *current*)

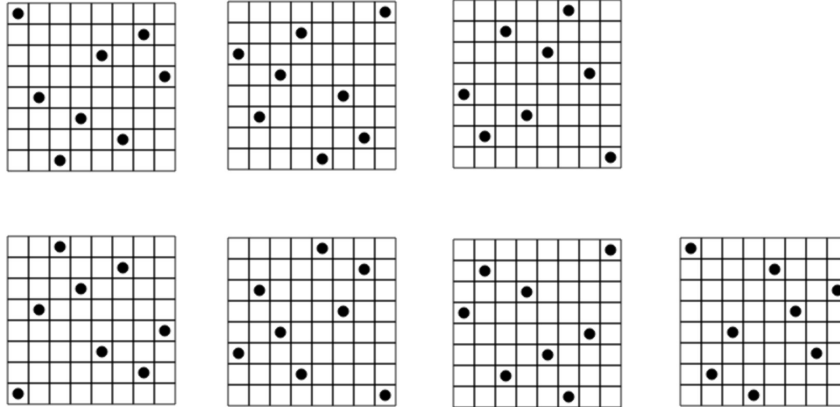
 set *var* = *value* in *current*

return *failure*

Symmetries

Idea: add constraints

Try remove them → Reduces search space, avoids exploring equivalent solutions



Symmetries

Idea: Add constraints to allow one solution per group of symmetric solutions

- only keep lexicographically smallest solution

A	B	C
D	E	F
G	H	I

 $\preceq$

C	B	A
F	E	D
I	H	G

$$\begin{aligned}
 &A < C \quad \backslash / \quad (A = C \quad / \backslash \\
 &\quad (B < B \quad \backslash / \quad (B = B \quad / \backslash \\
 &\quad \quad (C < A \quad \backslash / \quad (C = A \quad / \backslash \\
 &\quad \quad \quad (D < F \quad \backslash / \quad \dots) \dots)
 \end{aligned}$$

Homeworks

Read Chapter 5.1, 5.2, 5.3, 5.4, 5.5.3